ASSIGNMENT 4 – CO4

DESIGNING REUSABLE CLASS HIERARCHIES

Course: Object Oriented Programming with C++ (DSA01)

Submitted by: J. Roopa Sai Divya

Register Number: 192524140

Department: Artificial Intelligence and Data Science

Institution: SIMATS Engineering, Chennai

Submitted to: _____Dr.Arunkumar Palthasar_____

Submission Date: __09-11-2026_____

GitHub Repository Link: https://github.com/roopajallu814-arch/C-DSA0104-Roopa-192524140-

TASK 1 – CLASS DESIGN REVIEW

1.1 Diagnose the Design

The given payroll program contains three separate classes: Professor, LabAssistant and GuestFaculty. These classes have several design problems.

Design deficiencies:

1. Repeated data members: The fields name, id and basic are repeated in Professor and LabAssistant.
2. Public data members: All data members are public, so they can be modified directly without validation.
3. Inconsistent naming: Professor and LabAssistant use name and id, whereas GuestFaculty uses nm and code.
4. Unused field: The da field is declared but is not used in the pay() calculation.
5. Magic number: The value 0.42 is directly written in the function. It should be represented using a named constant.
6. Code duplication: Similar salary calculations are repeated in different classes.

7. Difficult extension: Adding a new staff category requires modifying several parts of the program.
8. No common interface: All staff types cannot be processed easily using one common base-class pointer.

1.2 Repair the Design

The design can be improved by creating a common base class called Staff. The common information such as name and ID is stored in the base class. The derived classes calculate their salary according to their own requirements.

C++ Program:

TASK 1 – CLASS DESIGN REVIEW

1.1 Diagnose the Design

The given payroll program contains three separate classes: Professor, LabAssistant and GuestFaculty. These classes have several design problems.

Design deficiencies:

1. Repeated data members: The fields name, id and basic are repeated in Professor and LabAssistant.
2. Public data members: All data members are public, so they can be modified directly without validation.
3. Inconsistent naming: Professor and LabAssistant use name and id, whereas GuestFaculty uses nm and code.
4. Unused field: The da field is declared but is not used in the pay() calculation.
5. Magic number: The value 0.42 is directly written in the function. It should be represented using a named constant.
6. Code duplication: Similar salary calculations are repeated in different classes.
7. Difficult extension: Adding a new staff category requires modifying several parts of the program.
8. No common interface: All staff types cannot be processed easily using one common base-class pointer.

1.2 Repair the Design

The design can be improved by creating a common base class called Staff. The common information such as name and ID is stored in the base class. The derived classes calculate their salary according to their own requirements.

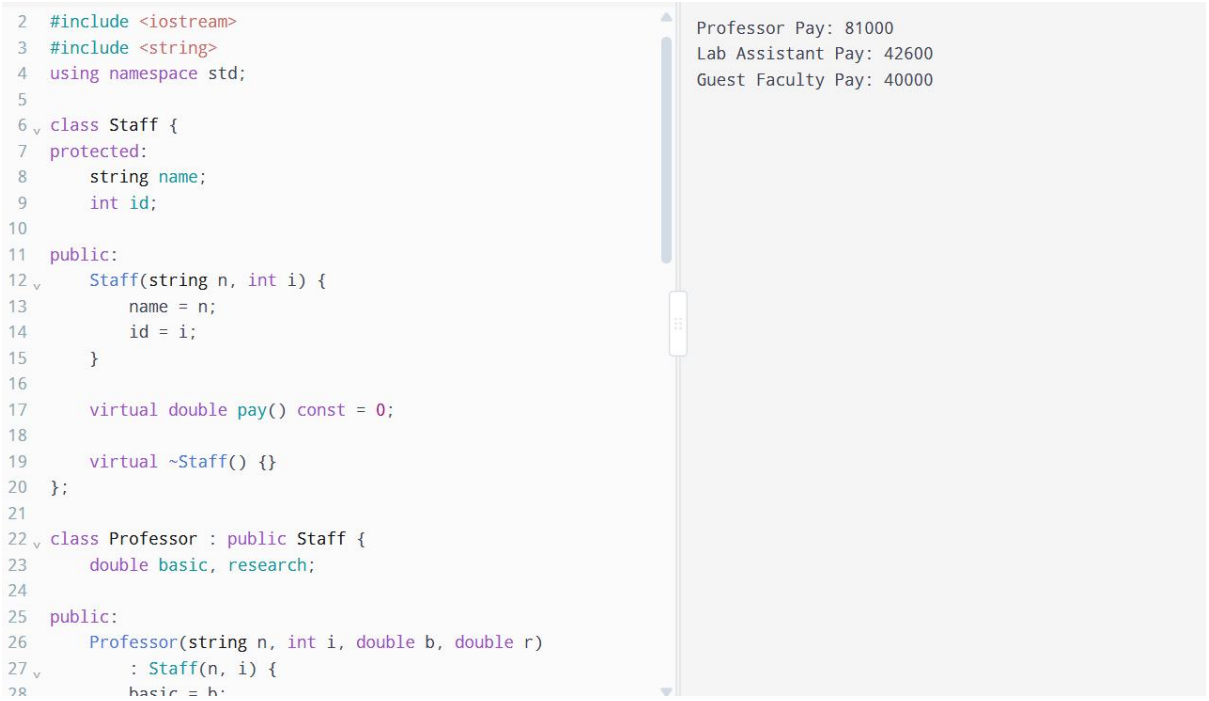
C++ Program:

```
#include <iostream>
#include <string>
using namespace std;
class Staff {
protected:
    string name;
    int id;
public:
    Staff(string n, int i) {
        name = n;
        id = i;
    }
    virtual double pay() const = 0;
    virtual void display() const {
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
    }
    virtual ~Staff() {}
};
class Professor : public Staff {
private:
    double basic, research;
public:
    Professor(string n, int i, double b, double r)
```

```
        : Staff(n, i) {
        basic = b;
        research = r;
    }
    double pay() const override {
        return basic + basic * 0.42 + research;
    }
};
class LabAssistant : public Staff {
private:
    double basic;
public:
    LabAssistant(string n, int i, double b)
        : Staff(n, i) {
        basic = b;
    }
    double pay() const override {
        return basic + basic * 0.42;
    }
};
class GuestFaculty : public Staff {
private:
    double hours, rate;
public:
    GuestFaculty(string n, int i, double h, double r)
        : Staff(n, i) {
        hours = h;
```

```
        rate = r;
    }
    double pay() const override {
        return hours * rate;
    }
};
int main() {
    Professor p("Anitha", 101, 50000, 10000);
    LabAssistant l("Kumar", 102, 30000);
    GuestFaculty g("Ravi", 103, 40, 1000);
    cout << "Professor Pay: " << p.pay() << endl;
    cout << "Lab Assistant Pay: " << l.pay() << endl;
    cout << "Guest Faculty Pay: " << g.pay() << endl;
    return 0;
}
```

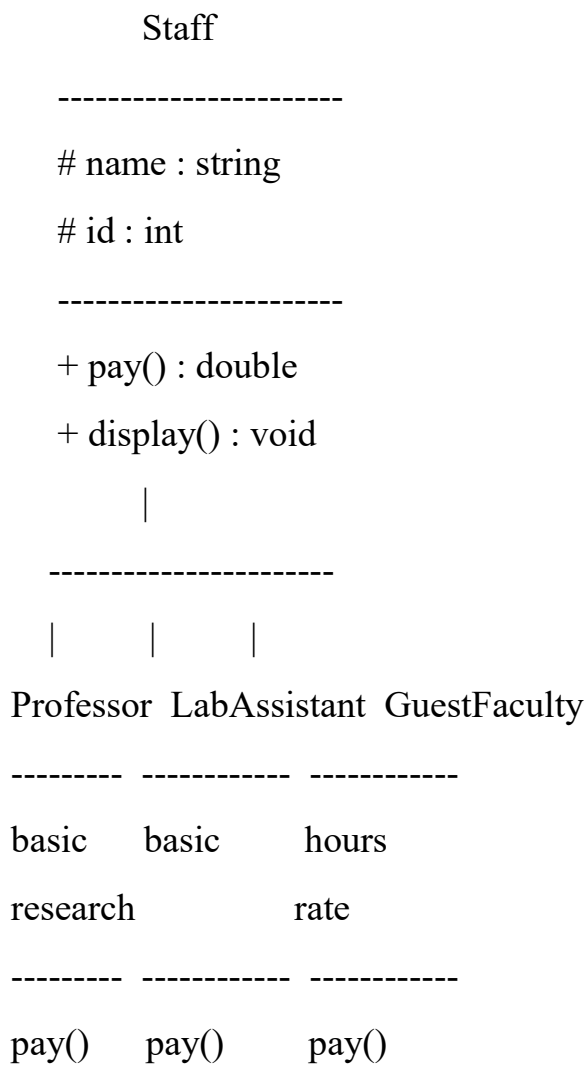
Output:



Explanation:

- Staff is the common base class.
- Professor, LabAssistant and GuestFaculty are derived classes.
- The pay() function is declared as a pure virtual function.
- Each derived class provides its own implementation of pay().
- Protected data members allow derived classes to access common information.

UML Class Diagram



1.3 Design Defence

The class hierarchy makes it easier to add a new staff category because all common data and functions are stored in the Staff base class. A new category can be created by inheriting from Staff and overriding the pay() function.

Existing classes do not need to be modified. This improves code reusability, maintainability and extensibility.

Alternative Concept and Complexity Analysis

An alternative design is to use one Staff class with a type field and a switch statement inside the pay() function.

In this design, the type field identifies whether the employee is a professor, lab assistant or guest faculty. The switch statement calculates the salary based on the employee type.

Comparison:

Feature	Inheritance	Single Staff Class
Pay calculation	Virtual function	Switch statement
Time per pay call	O(1)	O(1)
Space per object	Stores relevant fields	May contain unused fields
Adding a new type	Add a derived class	Modify the class and switch
Maintenance	Easier	More difficult

Conclusion: Inheritance is a better approach because it separates responsibilities and supports future extensions without changing existing classes.

TASK 2 – FROM REQUIREMENT TO HIERARCHY

2.1 Identify the Relationships

The university manages persons, students, staff, teaching assistants and research scholars.

The relationships are:

1. Person and Student: Student is a Person. This is single inheritance.
2. Person and Staff: Staff is a Person. This is hierarchical inheritance because multiple classes inherit from Person.
3. TeachingAssistant and Student: TeachingAssistant is a Student.
4. TeachingAssistant and Staff: TeachingAssistant is also a Staff member. This uses multiple inheritance.
5. ResearchScholar and Student: ResearchScholar is a Student. This uses multilevel inheritance.
6. Person and Address: A Person has an Address. This is composition (has-a relationship).

The complete design uses hybrid inheritance because it combines more than one type of inheritance.

2.2 Resolve the Diamond Problem

The diamond problem occurs when a class inherits from two classes that have the same base class.

In this example, TeachingAssistant inherits from both Student and Staff. Both classes inherit from Person.

Without virtual inheritance, TeachingAssistant receives two separate copies of Person. This creates ambiguity when accessing common members such as name and ID.

Solution:

Virtual inheritance is used to share only one common Person base-class object.

```
class Student : virtual public Person {

};

class Staff : virtual public Person {

};

class TeachingAssistant : public Student, public Staff {

};
```

Explanation: The virtual keyword ensures that TeachingAssistant contains only one shared Person base-class subobject.

2.3 Design the Classes

C++ Program:

```
#include <iostream>

#include <string>

using namespace std;

class Person {

protected:

    string name;
```

```
    int id;
    string phone;
public:
    Person(string n, int i, string p)
        : name(n), id(i), phone(p) {}
   void display() const {
        cout << name << " " << id << " " << phone << endl;
    }
};
class Student : virtual public Person {
protected:
    string programme;
    double cgpa;
public:
    Student(string n, int i, string p,
            string pr, double c)
        : Person(n, i, p),
          programme(pr), cgpa(c) {}
};
class Staff : virtual public Person {
protected:
    string department;
    double salary;
public:
    Staff(string n, int i, string p,
          string d, double s)
        : Person(n, i, p),
```

```
        department(d), salary(s) {}
};
class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int i, string p,
                      string pr, double c,
                      string d, double s)
        : Person(n, i, p),
          Student(n, i, p, pr, c),
          Staff(n, i, p, d, s) {}
};
class ResearchScholar : public Student {
private:
    double fellowship;
public:
    ResearchScholar(string n, int i, string p,
                    string pr, double c, double f)
        : Person(n, i, p),
          Student(n, i, p, pr, c),
          fellowship(f) {}
    void displayScholar() const {
        display();
        cout << "Fellowship: " << fellowship << endl;
    }
};
int main() {
    TeachingAssistant ta(
```

```
        "Roopa", 101, "9876543210",
        "AI and DS", 8.5, "CSE", 30000
    );
    cout << "Teaching Assistant Details" << endl;
    ta.display();
    ResearchScholar rs(
        "Kavya", 102, "9876501234",
        "CSE", 9.0, 15000
    );
    cout << "\nResearch Scholar Details" << endl;
    rs.displayScholar();
   return 0;
}
```

Output:

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name;
    int id;
    string phone;

public:
    Person(string n, int i, string p)
        : name(n), id(i), phone(p) {}

    void display() const {
        cout << name << " " << id << " " << phone << endl;
    }
};

class Student : virtual public Person {
protected:
    string programme;
    double cgpa;

public:
    Student(string n, int i, string p,
            string pr, double c)
```

```
Teaching Assistant Details
Roopa 101 9876543210

Research Scholar Details
Kavya 102 9876501234
Fellowship: 15000
```

2.4 Access Table

The following table shows how Person members are accessed in Student under different inheritance types.

Member in Person	Member in Person	Member in Person	Member in Person
Public Inheritance	Public Inheritance	Public Inheritance	Public Inheritance
Protected Inheritance	Protected Inheritance	Protected Inheritance	Protected Inheritance
Private Inheritance	Private Inheritance	Private Inheritance	Private Inheritance

Explanation:

- Public members remain public under public inheritance.
- Public and protected members become protected under protected inheritance.
- Public and protected members become private under private inheritance.
- Private members of the base class cannot be accessed directly by derived classes.

Predict the Output

Program:

```
#include <iostream>
using namespace std;
struct Person {
   Person() {
      cout << "Person ";
   }
};
struct Student : virtual Person {
   Student() {
      cout << "Student ";
   }
};
struct Staff : virtual Person {
   Staff() {
      cout << "Staff ";
   }
```

```
};
struct TeachingAssistant : Student, Staff {
    TeachingAssistant() {
        cout << "TA ";
    }
};
int main() {
    TeachingAssistant ta;
    cout << '\n';
}
```

Output:



Explanation:

1. Person is constructed first because it is the virtual base class.
2. Student is constructed next.
3. Staff is constructed after Student.
4. TeachingAssistant is constructed last.

Person is printed only once because virtual inheritance creates one shared Person base-class subobject.

Without Virtual Inheritance:

If the virtual keyword is removed, two Person subobjects are created. The output becomes:

Person Student Person Staff TA

There is no compilation error in this particular program because it does not access an ambiguous Person member.

Alternative Concept and Complexity Analysis

If TeachingAssistant contains Student and Staff objects as members instead of inheriting from both, it uses composition.

Example:

```
class TeachingAssistant {
   Student student;
   Staff staff;
};
```

In this design, TeachingAssistant has two separate member objects. Each member may contain its own Person information.

Feature	Virtual Inheritance	Composition
Relationship	Is-a	Has-a
Person objects	One shared base subobject	Usually two separate Person subobjects
Access	Through inheritance	Through member objects
Ambiguity	Avoided using virtual base	Access through named members
Space	Shared base object	Two member objects

Conclusion: Virtual inheritance is suitable when TeachingAssistant must represent both a Student and a Staff member while sharing one Person identity.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES

3.1 Pass Values to Base Constructors

A derived-class constructor can call its base-class constructor using a member initializer list.

In the university people management system, Student inherits from Person, Staff inherits from Person, and TeachingAssistant inherits from both Student and Staff.

Example:

```
class Person {
protected:
```

```
    string name;
    int age;
public:
    Person(string n, int a)
        : name(n), age(a) {}
};
class Student : virtual public Person {
public:
    Student(string n, int a)
        : Person(n, a) {}
};
class Staff : virtual public Person {
public:
    Staff(string n, int a)
        : Person(n, a) {}
};
class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int a)
        : Person(n, a),
          Student(n, a),
          Staff(n, a) {}
};
```

Why must the most-derived class call the virtual base constructor directly?

When virtual inheritance is used, the most-derived class is responsible for constructing the virtual base class. Therefore, TeachingAssistant must directly call the Person constructor. The constructor calls written inside Student and Staff are ignored for virtual-base initialization when a TeachingAssistant object is created.

3.2 Member Classes

A member class is an object declared inside another class. It represents a has-a relationship.

For example:

- Person has an Address.
- Student has a Course.

Example:

```
#include <iostream>
#include <string>
using namespace std;
class Address {
    string city;
    int pin;
public:
    Address(string c, int p)
        : city(c), pin(p) {}

    void display() const {
        cout << city << "-" << pin << endl;
    }
};
class Course {
    string courseName;
public:
    Course(string c)
        : courseName(c) {}
    void display() const {
        cout << courseName << endl;
```

```
    }
};
class Person {
protected:
    string name;
    Address home;
public:
    Person(string n, string city, int pin)
        : name(n), home(city, pin) {}
};
class Student : public Person {
    Course course;
public:
    Student(string n, string city, int pin, string c)
        : Person(n, city, pin), course(c) {}
   void display() const {
        cout << "Name: " << name << endl;
        cout << "Address: ";
        home.display();
       cout << "Course: ";
        course.display();
    }
};
int main() {
    Student s("Kavya", "Salem", 636001, "C++");
    s.display();
    return 0;
```

}

Output:



```
#include <iostream>
#include <string>
using namespace std;

class Address {
    string city;
    int pin;

public:
    Address(string c, int p)
        : city(c), pin(p) {}

    void display() const {
        cout << city << "-" << pin << endl;
    }
};

class Course {
    string courseName;

public:
    Course(string c)
        : courseName(c) {}

    void display() const {
        cout << courseName << endl;
```

```
Name: Kavya
Address: Salem-636001
Course: C++
```

3.3 Trace the Order of Construction and Destruction

Program:

```
#include <iostream>
using namespace std;
struct Engine {
    Engine() {
        cout << "Engine ";
    }
    ~Engine() {
        cout << "~Engine ";
    }
};
struct Vehicle {
    Vehicle() {
        cout << "Vehicle ";
    }
   ~Vehicle() {
```

```
        cout << "~Vehicle ";
    }
};
struct Car : Vehicle {
    Engine e;
    Car() {
        cout << "Car ";
    }
    ~Car() {
        cout << "~Car ";
    }
};
int main() {
    {
        Car c;
        cout << "| ";
    }
  cout << '\n';
}
```

Output:

```
    }

    ~Vehicle() {
        cout << "~Vehicle ";
    }
};
```

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Explanation:

Construction happens in the following order:

1. Base class Vehicle is constructed first.
2. Member object Engine is constructed next.

3. Derived class Car constructor executes last.

Destruction happens in the reverse order:

1. Car destructor executes first.
2. Engine member object is destroyed next.
3. Vehicle base class is destroyed last.

Rule: Base classes are constructed before derived classes. Member objects are constructed in the order in which they are declared. Destruction takes place in the reverse order.

3.4 Experiment – Remove Virtual Inheritance

When the virtual keyword is removed from the inheritance declarations, Student and Staff each contain a separate Person subobject.

Changes:

```
class Student : public Person {

};

class Staff : public Person {

};
```

Observation:

- Two Person subobjects are created inside TeachingAssistant.
- The Person constructor is called twice when a TeachingAssistant object is constructed.
- Accessing a Person member directly through TeachingAssistant may become ambiguous.
- The size of TeachingAssistant may increase because it contains two Person subobjects.

The exact sizeof(TeachingAssistant) value depends on the compiler, platform and class layout. It should be measured using sizeof() on the system used for execution.

Complete the Code – TODO 1

Aim: To initialize the base class and Address member using a member initializer list.

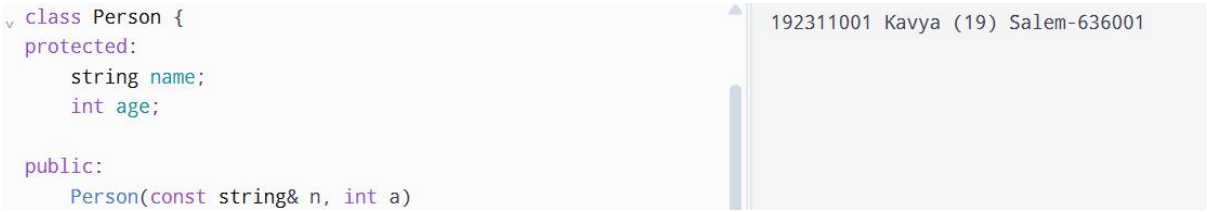
Completed C++ Program:

```
#include <iostream>
#include <string>
using namespace std;
class Person {
protected:
    string name;
    int age;
public:
    Person(const string& n, int a)
        : name(n), age(a) {}
};
class Address {
    string city;
    int pin;
public:
    Address(const string& c, int p)
        : city(c), pin(p) {}
   string str() const {
        return city + "-" + to_string(pin);
    }
};
class Student : public Person {
    long long regNo;
    Address home;
public:
    Student(const string& n, int a, long long r,
```

```
        const string& c, int p)
    : Person(n, a), regNo(r), home(c, p) {}
  void print() const {
    cout << regNo << ' ' << name << " ("
         << age << ") " << home.str() << '\n';
  }
};
int main() {
  Student s("Kavya", 19, 192311001, "Salem", 636001);
  s.print();

  return 0;
}
```

Output:



Explanation:

- Person(n, a) initializes the base class.
- regNo(r) initializes the registration number.
- home(c, p) initializes the Address member object.
- The initializer list avoids unnecessary default construction followed by assignment.

Initializer List Order Experiment

The initialization order is determined by the order in which members are declared in the class, not by the order in which they appear in the initializer list.

If the initializer list order is changed, the compiler may show a -Wreorder warning when compiled with -Wall -Wextra.

Conclusion: Members should be initialized in the same order in which they are declared.

Alternative Concept and Complexity Analysis

If Student stores an Address pointer instead of an Address object, dynamic memory allocation is required.

Example:

```
Address* home;

Student(const string& n, int a, const string& c, int p)
   : Person(n, a) {
   home = new Address(c, p);
}
```

The destructor must release the allocated memory:

```
~Student() {
   delete home;
}
```

The class must also carefully implement copying. A copy constructor and copy-assignment operator are needed to avoid two Student objects owning and deleting the same Address object.

Feature	Feature	Feature
Address Member Object	Address Member Object	Address Member Object
Address Pointer	Address Pointer	Address Pointer
Memory allocation	Memory allocation	Memory allocation
Automatic	Automatic	Automatic
Dynamic	Dynamic	Dynamic

Conclusion: A direct Address member is simpler and safer when every Student must have an Address. A pointer is useful when the Address is optional or needs a separately managed lifetime.

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS

4.1 Locate the Defects

The given bank account program contains three major problems.

1. Function hiding instead of runtime polymorphism

The rate() function in the base class is not virtual. Therefore, calling rate() through an Account pointer uses the base-class function even when the pointer refers to a SeniorAccount object.

2. Memory leak

The SeniorAccount class dynamically allocates an array using:

double* history = new double[100];

However, the base-class destructor is not virtual. Deleting a SeniorAccount object through an Account pointer causes undefined behaviour. The derived destructor may not run, so the allocated memory may not be released correctly.

3. Object slicing

The statement:

Account copy = SeniorAccount();

copies only the Account base-class portion into the Account object. The SeniorAccount-specific part is sliced away.

4.2 Construct Evidence

The following compiler command enables AddressSanitizer:

g++ -std=c++17 -Wall -Wextra -fsanitize=address -g main.cpp -o main

Run the program using:

./main

The original program has undefined behaviour when a derived object is deleted through a base pointer without a virtual destructor. AddressSanitizer may report the problem, depending on the compiler and runtime.

Expected evidence to record:

- Add a message inside both destructors.
- Observe whether the SeniorAccount destructor executes.
- Check the sanitizer report for memory errors.
- Save the actual output and sanitizer report screenshot.

Note: The exact sanitizer output depends on the compiler and execution environment. Do not copy an assumed sanitizer report as if it were your actual result.

4.3 Repair the Program

C++ Program:

```
#include <iostream>
using namespace std;
class Account {
public:
    virtual double rate() const {
        return 3.5;
    }
    virtual ~Account() {
        cout << "Account destroyed" << endl;
    }
};
class SeniorAccount : public Account {
private:
    double* history;
public:
    SeniorAccount() {
        history = new double[100];
    }
    double rate() const override {
        return 4.0;
    }
    ~SeniorAccount() override {
        delete[] history;
        cout << "SeniorAccount destroyed" << endl;
    }
```

```
};
int main() {
    Account* accounts[2];
    accounts[0] = new Account();
    accounts[1] = new SeniorAccount();
    for (Account* a : accounts) {
        cout << "Interest Rate: " << a->rate() << "%" << endl;
    }
    for (Account* a : accounts) {
        delete a;
    }
    return 0;
}
```

Output:

```
class Account {
public:
    virtual double rate() const {
        return 3.5;
    }

    virtual ~Account() {
        cout << "Account destroyed" << endl;
    }
};
```

```
Interest Rate: 3.5%
Interest Rate: 4%
Account destroyed
SeniorAccount destroyed
Account destroyed
```

Explanation:

- The virtual keyword enables runtime polymorphism.
- The override keyword confirms that the derived function overrides the base function.
- The virtual destructor ensures that the derived destructor is called when an object is deleted through a base-class pointer.
- The delete[] statement releases the dynamically allocated array.

4.4 Prevent Recurrence

Two important rules can prevent these defects:

1. If a base class is designed to be used polymorphically, it should have a virtual destructor.
2. Use the override keyword when redefining virtual functions in derived classes.

It is also better to use smart pointers such as std::unique_ptr instead of manually managing dynamic memory.

Predict the Output

Program:

```
#include <iostream>
using namespace std;
struct Account {
   double rate() const {
      return 3.5;
   }
   virtual double vrate() const {
      return 3.5;
   }
   virtual ~Account() = default;
};
struct SeniorAccount : Account {
   double rate() const {
      return 4.0;
   }
   double vrate() const override {
      return 4.0;
   }
};
```

```
int main() {
    SeniorAccount s;
    Account* p = &s;
    Account copy = s;
    cout << p->rate() << ' '
         << p->vrate() << ' '
         << copy.vrate() << '\n';
}
```

Output:

```
struct Account {
    double rate() const {
        return 3.5;
    }
```

```
3.5 4 3.5
```

Explanation:

- p->rate() prints 3.5 because rate() is not virtual. The base-class function is selected using the pointer's declared type.
- p->vrate() prints 4 because vrate() is virtual. Runtime polymorphism calls the SeniorAccount version.
- copy.vrate() prints 3.5 because copying SeniorAccount into Account causes object slicing. The copied object is only an Account.

Alternative Concept and Complexity Analysis

Instead of using raw pointers, accounts can be stored using std::unique_ptr<Account> in a vector.

Example:

```
#include <iostream>
#include <memory>
#include <vector>
using namespace std;
class Account {
public:
```

```
    virtual double rate() const {
        return 3.5;
    }
    virtual ~Account() = default;
};
class SeniorAccount : public Account {
public:
    double rate() const override {
        return 4.0;
    }
};
int main() {
    vector<unique_ptr<Account>> accounts;
    accounts.push_back(make_unique<Account>());
    accounts.push_back(make_unique<SeniorAccount>());
    for (const auto& a : accounts) {
        cout << a->rate() << endl;
    }
    return 0;
}
```

Comparison:

- Both approaches require O(n) time to process n accounts.
- Both use O(n) storage for n account objects and their ownership handles.
- Smart pointers automatically release memory when the vector is destroyed.
- Runtime polymorphism still works.
- Object slicing can still happen if objects are copied into a plain vector<Account>.

Conclusion: Using virtual functions and smart pointers makes the account system safer and easier to maintain.

TASK 5 – POINTERS TO OBJECTS AND THE THIS POINTER

5.1 Four Ways to Access an Object

An object can be accessed in four different ways.

1. Directly by object name

Cart c;

c.total();

2. Through a pointer

Cart* p = &c;

p->total();

3. Through a reference

Cart& r = c;

r.total();

4. Through dynamic memory allocation

Cart* p = new Cart();

p->total();

delete p;

The this pointer refers to the current object. It can be used to return the current object using *this and to distinguish a class member from a parameter with the same name.

5.2 Change the Requirement – Array of Pointers

An array of pointers can store the addresses of different Cart objects. The program can find the cart with the highest total by checking every pointer.

C++ Program:

#include <iostream>

using namespace std;

class Cart {

```
private:
    double amount;
public:
    Cart(double a) {
        amount = a;
    }
    double total() const {
        return amount;
    }
};
int main() {
    Cart c1(250);
    Cart c2(500);
    Cart c3(350);
    Cart* carts[3] = {&c1, &c2, &c3};
    Cart* highest = carts[0];
    for (int i = 1; i < 3; i++) {
        if (carts[i]->total() > highest->total()) {
            highest = carts[i];
        }
    }
    cout << "Highest Cart Total: "
         << highest->total() << endl;
    return 0;
}
```

Output:

```
class Cart {
private:
    double amount;

public:
    Cart(double a) {
        amount = a;
    }

    double total() const {
        return amount;
```

```
Highest Cart Total: 500
```

Explanation:

- An array of pointers stores the addresses of Cart objects.
- The arrow operator -> accesses members through a pointer.
- The loop compares the total of each cart.
- The pointer highest stores the address of the cart with the highest total.

5.3 Comparison of Four Ways

Method	Safety	Lifetime	Usage
Direct object	Generally safe	Scope-based	Simple object access
Pointer	Can be null	Depends on pointed object	Optional access and polymorphism
Reference	Must refer to an object	Depends on original object	Safe alias
Dynamic pointer	Needs memory management	Until delete	Dynamic lifetime

Why is this a pointer?

The this pointer stores the address of the current object. It allows a member function to access the current object's members.

What does *this refer to?

*this refers to the current object itself. Returning *this by reference allows method chaining.

Complete the Code – TODO 1 and TODO 2

Aim: To use the this pointer for method chaining and to distinguish a member variable from a parameter.

Completed C++ Program:

#include <iomanip>

```
#include <iostream>
#include <string>
using namespace std;
class Cart {
    string items[10];
    double prices[10];
    int n = 0;
    double discount = 0;
public:
    Cart& add(string item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            n++;
        }
        return *this;
    }
    Cart& setDiscount(double discount) {
        this->discount = discount;
        return *this;
    }
    double total() const {
        double t = 0;
        for (int i = 0; i < n; ++i) {
            t += prices[i];
        }
        return t * (100 - discount) / 100;
```

```
    }
};
int main() {
    Cart c;
    c.add("Rice", 60)
     .add("Dal", 120)
     .setDiscount(10);
    Cart* p = &c;
    p->add("Oil", 180);
    cout << fixed << setprecision(2)
         << p->total() << '\n';
    return 0;
}
```

Output:

```
class Cart {
    string items[10];
    double prices[10];
    int n = 0;
    double discount = 0;

public:
    Cart& add(string item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            n++;
        }

        return *this;
```

```
324.00
```

Explanation:

- add() stores the item and price when space is available.
- return *this returns the current object by reference.
- setDiscount() uses this->discount to refer to the data member.

- Method chaining allows multiple function calls in one statement.
- The final total is 360 minus 10% discount, which gives 324.

What Happens if add() Returns Cart Instead of Cart&?

If add() returns Cart by value, a copy of the object is returned instead of a reference to the original object.

This can cause unnecessary copying. Method chaining may continue on temporary copies rather than directly modifying the original object. Returning Cart& is more efficient because it returns a reference to the same object.

Alternative Concept and Complexity Analysis

If carts are stored in a std::vector<Cart> instead of an array of pointers, the vector directly stores Cart objects.

Example:

vector<Cart> carts;

carts.push_back(Cart());

Comparison:

Feature	Array of Pointers	Vector of Objects
Stored value	Object addresses	Actual objects
Access	Through ->	Through .
Search highest total	O(n)	O(n)
Storage	Objects elsewhere plus pointers	Objects inside vector
Memory management	Must manage pointed objects	Automatic
Reallocation	Pointer array itself does not move pointed objects	Vector growth may invalidate pointers and references to its elements

Conclusion: A vector of objects is generally simpler when the carts do not need independent dynamic lifetimes. An array of pointers is useful when objects are managed separately or polymorphism is required.

TASK 6 – DESIGN CHALLENGE

Vehicle Rental Fleet

Stage 1 – Interpret the Problem

The vehicle rental company rents different types of vehicles, including bicycles, two-wheelers, cars and electric cars.

Entities and their data:

Entity	Data Members	Behaviour
Vehicle	Registration number, daily rate	Calculate rental charge
Bicycle	Registration number, daily rate	Calculate bicycle rental charge
TwoWheeler	Fuel level, service due date	Calculate rental charge
Car	Seating capacity	Calculate car rental charge
ElectricCar	Battery capacity, charge level	Calculate charge based on kWh

Requirements as testable statements:

1. The system must store the registration number of every vehicle.
2. Each vehicle must have a daily rental rate.
3. The system must calculate rental charges according to the vehicle type.
4. Motor vehicles must store fuel or charge level and service due date.
5. Cars must store seating capacity.
6. Electric cars must store battery capacity.
7. The system must support renting and returning vehicles.
8. The system must calculate additional charges for late returns.
9. New vehicle types must be added without modifying existing vehicle classes.

Stage 2 – Propose the Hierarchy

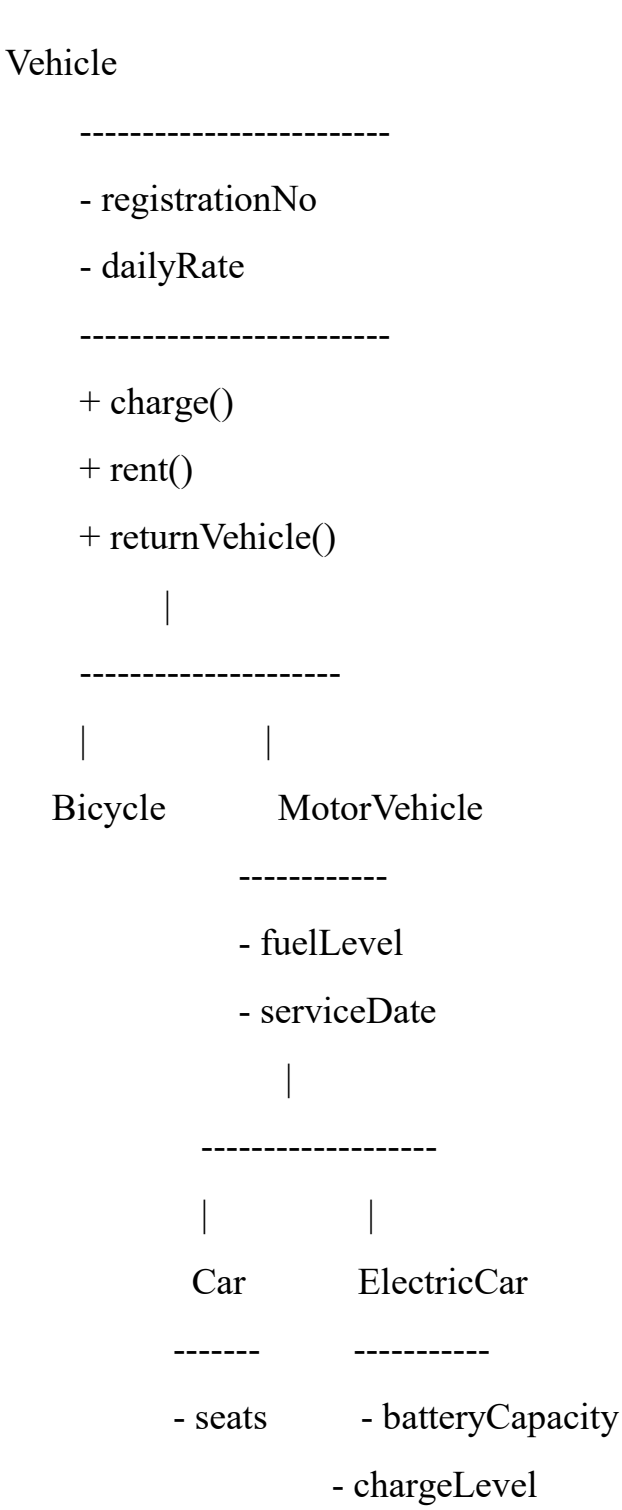
An abstract base class called Vehicle is created to store common information.

The derived classes are:

- Bicycle
- MotorVehicle
- Car
- ElectricCar

MotorVehicle can be used as a common base class for vehicles with fuel or charge information. Car and ElectricCar can inherit from it.

UML Class Diagram:



Types of Inheritance:

- Bicycle uses single inheritance from Vehicle.
- MotorVehicle uses single inheritance from Vehicle.

- Car uses multilevel inheritance through MotorVehicle.
- ElectricCar uses multilevel inheritance through MotorVehicle.
- The overall hierarchy uses hierarchical inheritance.

Stage 3 – Identify Critical Design Decisions

1. Common data in Vehicle

The registration number and daily rate are stored in the base class because all vehicles share these properties.

2. Abstract class

The Vehicle class is an abstract class because every vehicle must calculate its own rental charge.

3. Virtual function

The charge() function is declared as pure virtual. Every derived class provides its own implementation.

4. Virtual destructor

A virtual destructor is used so that derived-class destructors are called correctly when objects are deleted through base-class pointers.

5. Fleet storage

An array of base-class pointers is used to store different vehicle types together.

Stage 4 – Design the Classes

C++ Program:

```
#include <iostream>
#include <string>
using namespace std;
class Vehicle {
protected:
    string registrationNo;
    double dailyRate;
public:
    Vehicle(string reg, double rate)
```

```
        : registrationNo(reg), dailyRate(rate) {}
    virtual double charge(int days) const = 0;
    virtual void display() const {
        cout << "Registration Number: "
             << registrationNo << endl;
    }
    virtual ~Vehicle() {}
};
class Bicycle : public Vehicle {
public:
    Bicycle(string reg, double rate)
        : Vehicle(reg, rate) {}
    double charge(int days) const override {
        return dailyRate * days;
    }
};
class MotorVehicle : public Vehicle {
protected:
    double fuelLevel;
    string serviceDate;
public:
    MotorVehicle(string reg, double rate,
                 double fuel, string date)
        : Vehicle(reg, rate),
          fuelLevel(fuel), serviceDate(date) {}
};
class Car : public MotorVehicle {
```

```
private:
    int seats;
public:
    Car(string reg, double rate, double fuel,
        string date, int s)
        : MotorVehicle(reg, rate, fuel, date),
          seats(s);
    double charge(int days) const override {
        return dailyRate * days;
    }
    void display() const override {
        Vehicle::display();
        cout << "Seats: " << seats << endl;
    }
};
class ElectricCar : public MotorVehicle {
private:
    double batteryCapacity;
    double chargeLevel;
public:
    ElectricCar(string reg, double rate, double fuel,
                string date, double battery, double level)
        : MotorVehicle(reg, rate, fuel, date),
          batteryCapacity(battery),
          chargeLevel(level) {}
    double charge(int days) const override {
        return dailyRate * days
```

```
            + batteryCapacity * 2;
    }  void display() const override {
        Vehicle::display();
        cout << "Battery Capacity: "
            << batteryCapacity << " kWh" << endl;
        cout << "Charge Level: "
            << chargeLevel << "%" << endl;
    }
};
int main() {
    Bicycle b("BIKE101", 100);
    Car c("CAR102", 1500, 80, "2026-12-01", 5);
    ElectricCar e("EV103", 2000, 0,
                "2026-12-05", 60, 90);
    Vehicle* fleet[3] = {&b, &c, &e};
    int days = 2;
    for (int i = 0; i < 3; i++) {
        fleet[i]->display();
        cout << "Rental Charge: Rs. "
            << fleet[i]->charge(days) << endl;
        cout << "--------------------" << endl;
    }
    return 0;
}
```

Output:

```
class Vehicle {
protected:
    string registrationNo;
    double dailyRate;

public:
    Vehicle(string reg, double rate)
        : registrationNo(reg), dailyRate(rate) {}

    virtual double charge(int days) const = 0;

    virtual void display() const {
        cout << "Registration Number: "
```

```
Registration Number: BIKE101
Rental Charge: Rs. 200
------------------
Registration Number: CAR102
Seats: 5
Rental Charge: Rs. 3000
------------------
Registration Number: EV103
Battery Capacity: 60 kWh
Charge Level: 90%
Rental Charge: Rs. 4120
------------------
```

Explanation:

- The Vehicle class stores common information.
- The Bicycle, Car and ElectricCar classes calculate their charges independently.
- The charge() function is a pure virtual function.
- The array fleet stores pointers to different vehicle objects.
- Runtime polymorphism calls the correct derived-class function.

Pseudocode for Renting and Returning a Vehicle

Algorithm: Rent Vehicle

1. Start.
2. Read the registration number of the vehicle.
3. Search for the vehicle in the fleet.
4. If the vehicle is available:
    - Mark the vehicle as rented.
    - Record the rental date.
    - Display the rental confirmation.
5. Otherwise, display that the vehicle is unavailable.
6. Stop.

Algorithm: Return Vehicle

1. Start.
2. Read the vehicle registration number.

3. Search for the rented vehicle.
4. Read the return date.
5. Calculate the number of rental days.
6. Calculate the rental charge using the vehicle's charge() function.
7. If the vehicle is returned late, calculate the late fee.
8. Display the final amount.
9. Mark the vehicle as available.
10. Stop.

Stage 5 – Implement and Demonstrate

The above program demonstrates the rental charge calculation for three vehicle types.

Demonstration of returning with a late charge:

Suppose a car has a daily rental rate of Rs. 1,500.

- Normal rental period: 2 days
- Daily rental charge: Rs. 1,500
- Normal rental amount: Rs. 3,000
- Late return charge: Rs. 500

Total amount:

\[3000 + 500 = 3500 \]

Final Amount: Rs. 3,500

Note: The sample program above demonstrates charge calculation. The late-return calculation, vehicle availability management, and adding a new electric bus are additional implementation requirements and should be included in the final extended program.

Adding a New Vehicle Type – Electric Bus

A new vehicle type can be added by creating a class that inherits from Vehicle.

```
class ElectricBus : public Vehicle {
public:
    ElectricBus(string reg, double rate)
```

```
    : Vehicle(reg, rate) {}

  double charge(int days) const override {
    return dailyRate * days + 500;
  }
};
```

The existing Vehicle, Bicycle, Car and ElectricCar classes do not need to be modified.

This demonstrates extensibility through inheritance and polymorphism.

Alternative Concept and Complexity Analysis

An alternative design is to use a single Vehicle class with a type code and a switch statement inside the charge() function.

Feature	**Abstract Base Class**	**Single Vehicle Class**
Charge calculation	Virtual function	Switch statement
Time per charge call	O(1)	O(1)
Space per object	Common fields and object-specific fields	May contain unused fields
Adding new type	Add a new derived class	Modify the switch statement
Maintainability	Easier	More difficult
Extensibility	High	Lower

Conclusion: The abstract base-class approach is better because it supports runtime polymorphism, code reusability and easy extension. New vehicle types can be added without modifying the existing classes.